

一、数据清洗和特征处理方法

1.1 数据探索与特征剔除

原始数据集经过分析，剔除了以下对预测结果无益或缺失值过多的特征：

PassengerId：唯一标识符，与生存率无关。

Name / Ticket：字符串类型且无规律，难以直接转换为数值特征。

Cabin：缺失值超过 70%，有效信息极少，保留反而引入噪声。

1.2 缺失值处理

Age (年龄)：使用训练集的中位数填充。中位数相比均值更具抗异常值能力。

Fare (票价)：使用训练集的中位数填充。

Embarked (登船港口)：仅少量缺失，使用训练集的众数（出现次数最多的值）填充。

1.3 类别特征转换与数值标准化

类别编码：Sex 映射为 male: 0, female: 1；Embarked 映射为 S: 0, C: 1, Q: 2。

数值标准化 (Z-score)：对 Age 和 Fare 进行标准化，公式为 $(x - \text{mean}) / \text{std}$ 。

注意事项：标准化所需的均值和标准差仅从训练集计算，验证集、测试集和新样本必须复用该套参数，防止数据泄露。

二、模型结构、损失函数、优化器及主要超参数

2.1 模型结构

模型定义如下：

继承 `nn.Module`，使用 `nn.Sequential` 构建多层感知机 (MLP)。

输入层：7 个神经元（对应 Pclass, Sex, Age, SibSp, Parch, Fare, Embarked）。

隐藏层：`nn.Linear(7, 16) + nn.ReLU()` 激活函数。

输出层：`nn.Linear(16, 1)`，输出 logits（未经过 Sigmoid 的原始值）。

2.2 损失函数

采用 `nn.BCEWithLogitsLoss()`。该损失函数内部整合了 Sigmoid 和交叉熵，相比手动 Sigmoid + BCELoss 在数值计算上更稳定。

2.3 优化器与超参数

优化器：`optim.Adam`，自适应学习率。

学习率：0.001。

Batch Size：32（批次增大可让梯度更新更稳定）。

Epochs：800 轮。

随机种子：`torch.manual_seed(42)` 和 `np.random.seed(42)`。

三、数据划分方式、随机种子和实验配置

3.1 数据划分

将 891 条数据按 **7 : 1.5 : 1.5** 的比例进行切分：

训练集 (70%)：用于模型参数更新。

验证集 (15%)：每轮训练结束后评估，用于监控模型是否过拟合，不参与梯度更新。

测试集 (15%)：训练全部结束后评估一次，作为最终评价指标。

划分时使用 `random_state=42`，确保每次运行划分结果一致。

3.2 实验配置

系统环境：Windows, Python 3.x, PyTorch。

设备：CPU 运行。

遇到的问题及解决方法

4.1 Loss 曲线剧烈震荡

问题：一开始打印 Loss 时曲线上下抖动严重。

原因：代码误将每个 Epoch 中最后一个 Batch 的 Loss 当作该轮的平均 Loss 记录，随机性极大。

解决：增加变量累加每一个 Batch 的 Loss，最后除以 `len(train_loader)` 计算真正的整轮平均 Loss，曲线立刻变得平滑。

测试集准确率、实验结果分析和心得体会

5.1 实验结果

最终测试集准确率：0.79.

Loss 曲线：随着 Epoch 增加稳步下降，最终趋于平稳。

Accuracy 曲线：准确率在早期快速上升，后期趋于平稳，没有出现明显的过拟合

5.2 心得与想法

本程序由本人经过B站学习、资料查阅之后，通过个人理解与整合而形成，学会了没尝试过的环境配置与模块调用，学习到了从没见过的python和pytorch的各种函数、np和pd、plt等工具及其作用；有些代码行个人是在综合了AI给出的方案之后再进行修改或使用的，并确保每一步都明白这么写的原因。

专注于提高准确率的方式，但后来意识到想达到80多及以上是困难的，因为历史噪声或者说运气因素影响的生还结果在数据集中是不可抹去的

注：运行脚本需在终端里输入

```
python main.py
```